

# OctoLoRA: Gradient-Adaptive Rebalancing of LoRA+ for Low-Rank Fine-Tuning

Fabio Ciraci

University of Bologna, DISI

`fabio.ciraci@unibo.it`

## Abstract

Low-Rank Adaptation (LoRA) trains a small update  $BA$  on top of a frozen weight matrix, but  $A$  and  $B$  do not train at compatible rates:  $B$ 's zero initialization leaves it undertrained relative to  $A$  under a shared learning rate. LoRA+ fixes this with a fixed learning-rate ratio between the two matrices. We introduce a simple, training-time-only modification — a per-layer gate that rescales the gradient reaching  $A$  based on an exponential moving average of  $A$ 's and  $B$ 's recent gradient norms — intended to make that ratio adaptive rather than fixed. The gate never changes the forward computation, so it adds zero inference cost and zero parameters. On GSM8K with Llama-3.1-8B-Instruct, we find the method's naive evaluation is misleading: at LoRA+'s commonly-cited default ratio (16), our combined method *loses* to plain LoRA by 3.8 percentage points, and this initially looked like the method didn't work at all. Diagnosing training loss against test accuracy revealed the LoRA+ variants were overfitting; sweeping the ratio down to 2 recovers and then exceeds plain LoRA's performance. At this corrected ratio, gate + LoRA+ beats plain LoRA by 1.3 percentage points on average, confirmed across three seeds with non-overlapping accuracy ranges. We report this result together with its full negative-result history, situate it against more principled recent fixes for the same A/B-imbalance problem (LoRA-RITE, ALLoRA) whose reported effect sizes are substantially larger, and are explicit about the narrow, hyperparameter-sensitive scope of what we show.

## 1 Introduction

Fine-tuning large language models by updating every parameter is expensive. LoRA [Hu et al., 2021] makes this cheap: freeze a pretrained weight matrix  $W$ , and learn a low-rank update  $\Delta W = BA$ , where  $A \in \mathbb{R}^{r \times d_{in}}$  and  $B \in \mathbb{R}^{d_{out} \times r}$  with rank  $r \ll \min(d_{in}, d_{out})$ . Only  $A$  and  $B$  are trained.

A known inefficiency in this recipe is that  $A$  and  $B$  do not train at compatible rates.  $A$  is randomly initialized and  $B$  is initialized to zero, and Hayou et al. [2024] show that under a shared learning rate this asymmetry causes  $B$  to be systematically undertrained as model width grows. Their fix, **LoRA+**, is a single global hyperparameter: give  $B$  a fixed learning-rate multiplier (16× by default) relative to  $A$ , for the entire training run.

This paper investigates a natural follow-up question: if a *fixed* ratio helps, does an *adaptive* one help more? We implement a per-layer gate that tracks an exponential moving average (EMA) of the gradient norms flowing into  $A$  and  $B$  separately, and uses their ratio to rescale how much of  $A$ 's gradient is allowed through at each training step — a dynamic, per-layer analogue of LoRA+'s fixed, global ratio. We call the resulting method **OctoLoRA**.

The empirical path to our result is unusually informative and we report it in full rather than only the final number. An initial ablation at LoRA+'s default ratio (16) showed the combined method losing to plain LoRA by 3.8 points — the opposite of our hypothesis. Rather than treat this as

a negative result, we diagnosed it: comparing training loss to test accuracy showed the LoRA+ variants were *overfitting*, and sweeping the ratio down revealed a clear, monotonic recovery. At a corrected ratio of 2, the full method beats plain LoRA by 1.3 points on average, confirmed across three random seeds with non-overlapping accuracy ranges. We consider this progression — wrong hyperparameter, apparent negative result, correct diagnosis, corrected and confirmed positive result — as important to the paper’s contribution as the final number, since it is a directly reproducible cautionary tale about drawing conclusions from a single, unswept hyperparameter setting.

We make three contributions:

1. A gradient-adaptive, per-layer alternative to LoRA+’s fixed learning-rate ratio, requiring no additional parameters and no change to the forward computation (Section 3).
2. An empirical demonstration on GSM8K fine-tuning of Llama-3.1-8B-Instruct that the method’s apparent failure at a commonly used hyperparameter setting is an artifact of that setting, not of the method, and that a corrected setting yields a real, seed-confirmed improvement (Section 5).
3. A comparison against concurrent, more principled solutions to the same problem (LoRA-RITE, ALLoRA) that situates our effect size honestly relative to the current state of the art, rather than only against LoRA and LoRA+ (Section 2).

## 2 Related Work

**LoRA and LoRA+.** LoRA [Hu et al., 2021] is the foundation this work builds on. LoRA+ [Hayou et al., 2024] is the method we directly modify: a fixed, global learning-rate ratio between  $A$  and  $B$ , motivated by a width-scaling analysis of feature learning. It is the correct baseline for our ablation, since our gate targets exactly the imbalance LoRA+ targets, only adaptively.

**Other adaptive fixes for the A/B imbalance.** We are not the only recent attempt to make this fix adaptive rather than fixed. **ALLoRA** [ALLoRA, 2024] scales per-row gradients inversely proportional to the  $\ell_2$  norm of that row of  $BA$ , an instantaneous (no-EMA) weight-magnitude-driven rule; it reports a modest but consistent average improvement (+0.3%) over LoRA and DoRA across several models. **LoRA-RITE** [LoRA-RITE, 2024] addresses a related but distinct issue — transformation invariance, the fact that many  $(A, B)$  pairs represent the same product  $BA$  with very different condition numbers — via optimizer-level matrix preconditioning derived from a polar decomposition. It is a substantially more principled and expensive method than ours, and reports large, verified gains: +7.13 points on GSM8K (Gemma-7B, 48.37%  $\rightarrow$  55.50% over plain Adam) and +8.2 points on a broader benchmark average. **Balanced LoRA (BaLoRA)** [BaLoRA, 2026] takes a third angle, projecting  $(A, B)$  onto a manifold that improves the pair’s condition number each step; its abstract claims faster convergence and superior performance but we could not verify specific benchmark numbers from the available text. None of these three methods use our specific construction (an EMA of gradient norms as a scalar gate on gradient flow into  $A$ ), but the problem space is considerably more active and, in LoRA-RITE’s case, considerably better-solved than a comparison against LoRA+ alone would suggest — we return to this in Section 6.

**FlyLoRA and the multi-task interference problem.** FlyLoRA [Zou et al., 2025] is a different problem behind a similar name: it addresses *inter-task* interference when multiple LoRA adapters are merged, using a frozen random sparse projection (inspired by the fly olfactory circuit) in place

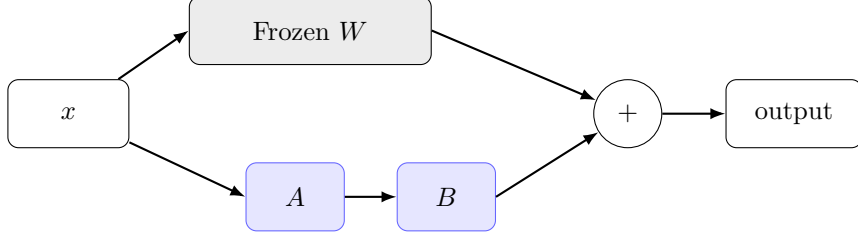


Figure 1: The forward pass:  $\text{output} = Wx + \frac{\alpha}{r} B(A(x))$ . This computation is identical for plain LoRA, LoRA+, and OctoLoRA in every case; the three methods differ only in how a training step updates  $A$  and  $B$  (Eq. 1–3), never in what the layer computes.

of a trainable down-projection, together with rank-wise expert activation in the up-projection — an implicit Mixture-of-Experts with no trainable router. Our method has no expert routing, no multi-adapter merging setting, and targets single-task training dynamics, not cross-task interference. We note this explicitly because our project’s working name was inspired by FlyLoRA’s biological framing; the two methods are otherwise unrelated, and FlyLoRA is not a directly competing baseline for what we evaluate here.

### 3 Method

For a target linear layer with frozen weight  $W$  (Figure 1), LoRA computes

$$\text{output} = Wx + \frac{\alpha}{r} BAx, \quad (1)$$

where  $A$  is initialized with a standard random scheme and  $B$  is initialized to zero, so training starts at  $\Delta W = 0$ . Only  $A$  and  $B$  are updated.

**LoRA+.** Hayou et al. [2024] put  $A$  and  $B$ ’s parameters into separate optimizer groups with a fixed ratio  $\rho$  between their learning rates:

$$\eta_B = \rho \cdot \eta_A, \quad \rho = \text{const. (default 16)}. \quad (2)$$

**OctoLoRA’s gate.** We replace the fixed constant  $\rho$  with a quantity recomputed every training step from the layer’s own recent gradient history. Each layer maintains an EMA of the gradient norms flowing into  $A$  and  $B$  (decay 0.9) via backward hooks, and computes

$$a_{\text{weight}} = \text{clip}\left(\frac{\|\nabla B\|_{\text{EMA}}}{\|\nabla A\|_{\text{EMA}} + \|\nabla B\|_{\text{EMA}}}, 0.1, 1.0\right), \quad (3)$$

$$\widetilde{A}(x) = a_{\text{weight}} \cdot A(x) + (1 - a_{\text{weight}}) \cdot \text{sg}[A(x)], \quad (4)$$

where  $\text{sg}[\cdot]$  is the stop-gradient operator, and  $\widetilde{A}(x)$  replaces  $A(x)$  in Eq. 1. Because  $a_{\text{weight}} \cdot A(x) + (1 - a_{\text{weight}}) \cdot \text{sg}[A(x)]$  is numerically equal to  $A(x)$  for any value of  $a_{\text{weight}}$  — the stop-gradient term only removes gradient flow, it does not change the value — **the forward computation is unaffected by the gate at every step, during both training and inference.**  $a_{\text{weight}}$  only rescales how much gradient reaches  $A$  during backpropagation. This is the precise sense in which OctoLoRA is *not* a routing or Mixture-of-Experts mechanism, despite drawing its name from that family of ideas: there is one  $A/B$  pair per layer, and the only effect of training with the gate is on which  $A, B$  values gradient descent converges to. The gate and LoRA+’s optimizer split (Eq. 2) are independent and can be toggled separately, which is what enables the ablation in Section 5.

Table 1: First ablation sweep,  $\rho = 16$  (LoRA+’s default), single seed. Plain LoRA outperforms every variant involving LoRA+ or the gate.

| Variant                     | Final train loss | GSM8K accuracy |
|-----------------------------|------------------|----------------|
| <code>vanilla_lora</code>   | 0.744            | <b>72.71%</b>  |
| <code>lora_plus_only</code> | 0.660            | 70.96%         |
| <code>gate_only</code>      | 0.744            | 71.72%         |
| <code>octo_lora_plus</code> | 0.662            | 68.92%         |

## 4 Experimental Setup

**Model and data.** Llama-3.1-8B-Instruct [AI@Meta, 2024], fine-tuned on GSM8K [Cobbe et al., 2021] (grade-school math word problems), formatted as single-turn chat completions (“Solve this step by step: {question}”). Training uses  $\sim 9.3$ k examples; evaluation uses the full 1,319-example test split.

**LoRA configuration.** Rank  $r = 16$ ,  $\alpha = 32$ , applied to the attention projections ( $q, k, v, o$ ; 128 layers replaced). 3 epochs, effective batch size 32 (micro-batch  $4 \times$  gradient accumulation 8), base learning rate  $2 \times 10^{-4}$ , cosine schedule, bf16.

**Evaluation.** Greedy decoding (temperature 0), up to 256 new tokens per question, exact match on the extracted final numeric answer, over the *full* 1,319-example test set. We evaluate on the full set rather than a subset because we found a 200-example subset can invert the sign of an effect relative to the full set (Section 5, Table 5) — a methodological finding we report separately from our main results because it generalizes beyond this specific method.

**Ablation design.** The gate and LoRA+’s optimizer split are independently toggleable, forming a  $2 \times 2$  design (`vanilla_lora`, `lora_plus_only`, `gate_only`, `octo_lora_plus`), which separates the main effect of LoRA+, the main effect of the gate, and their interaction, rather than only reporting whether the fully-combined method beats a single baseline. We additionally sweep LoRA+’s ratio  $\rho \in \{2, 4, 8, 16\}$  after the initial ablation revealed a strong dependence on this hyperparameter.

**Compute.** Single-GPU jobs (NVIDIA L40S), one run each unless noted as seed-repeated. All code, exact hyperparameters, and SLURM submission scripts are released at <https://github.com/FabioC-alt/OctoLoRA>.

## 5 Results

### 5.1 Initial ablation: an apparent failure at the default ratio

Table 1 shows the first ablation, run at LoRA+’s default ratio ( $\rho = 16$ ). Plain LoRA wins outright: LoRA+’s main effect is  $-2.28$  points, the gate’s main effect is  $-1.52$  points, and their combination ( $-3.79$  points) is worse than the sum of the two individual effects would predict ( $-2.75$  points) — an apparent negative interaction. Taken at face value, this contradicts our hypothesis entirely.

### 5.2 Diagnosis: overfitting from an over-aggressive ratio

Comparing training loss to test accuracy in Table 1 reveals the mechanism: the LoRA+ variants reach *lower* training loss than plain LoRA, yet score *worse* on held-out questions — a textbook overfitting signature. At  $\rho = 16$ ,  $B$ ’s effective learning rate is  $2 \times 10^{-4} \times 16 = 3.2 \times 10^{-3}$ , apparently too aggressive for this model/rank/task combination over only 3 epochs.

Table 2: Sweeping LoRA+’s ratio  $\rho$  (gate off). Accuracy is non-monotonic only within noise (Section 5.4); the trend across  $\rho = 2$  to 16 is a clear decline as  $\rho$  increases, consistent with progressively worse overfitting.

| $\rho$ | Final train loss | GSM8K accuracy | $\Delta$ vs. vanilla (72.71%) |
|--------|------------------|----------------|-------------------------------|
| 2      | 0.7246           | <b>73.24%</b>  | +0.53                         |
| 4      | 0.7038           | 72.25%         | −0.46                         |
| 8      | 0.6775           | 70.51%         | −2.20                         |
| 16     | 0.6600           | 70.96%         | −1.75                         |

Table 3: `octo_lora_plus` at the corrected ratio ( $\rho=2$ ) vs. `vanilla_lora`, three seeds each. Ranges do not overlap.

| Configuration                            | Seed 42 | Seed 123 | Seed 7 | Mean          |
|------------------------------------------|---------|----------|--------|---------------|
| <code>vanilla_lora</code>                | 72.71%  | 72.33%   | 71.27% | 72.10%        |
| <code>octo_lora_plus</code> ( $\rho=2$ ) | 73.31%  | 73.24%   | 73.62% | <b>73.39%</b> |

Table 2 confirms this directly: as  $\rho$  decreases from 16 toward 2, training loss rises back toward plain LoRA’s level and test accuracy recovers, eventually *exceeding* plain LoRA at  $\rho = 2$ . The conclusion from Table 1 was correct at  $\rho = 16$  but does not generalize to LoRA+ as a method — it was a statement about one hyperparameter setting.

### 5.3 Is the gate doing anything? Correcting an initial inference

An initial reading of Table 1 noted that `gate_only` and `vanilla_lora` have visually near-identical training loss curves step-for-step, which we initially (incorrectly) took as evidence the gate was inert. Direct instrumentation of  $a_{\text{weight}}$  (Eq. 3), logged across all 128 gated layers throughout training, corrects this: the mean of  $a_{\text{weight}}$  drifts from 0.85 to 0.79 over training, with per-layer values ranging from 0.58 to 0.95 — a real, non-trivial, layer-varying signal, not a value pinned at its ceiling. The gate is mechanistically active; its near-identical effect on the aggregate loss curve was a coincidence of this particular configuration, not evidence of inactivity, and is a caution against inferring an internal mechanism’s behavior only from aggregate outputs when it can be measured directly.

### 5.4 Noise floor

Re-running `gate_only` with an identical configuration and seed (only adding the logging above) gives 71.72% vs. 71.49% — a 0.23-point swing from pure run-to-run nondeterminism (bf16 training on GPU is not bit-reproducible even at a fixed seed). We treat differences below approximately 0.3–0.5 points as indistinguishable from this floor without further evidence.

### 5.5 The headline result, seed-confirmed

We repeated the corrected combined configuration (`octo_lora_plus`,  $\rho=2$ ) at two additional seeds, alongside matching `vanilla_lora` reruns (Table 3). The effect holds up cleanly: the combined method’s *worst* seed (73.24%) still beats plain LoRA’s *best* seed (72.71%) — the three-seed ranges do not overlap. The mean gap is +1.29 points, larger than the original single-seed estimate (+0.60),

Table 4: The gate’s effect on top of LoRA+ alone, at each ratio tested.

| $\rho$ | LoRA+ alone | Gate + LoRA+ | Gate’s added effect |
|--------|-------------|--------------|---------------------|
| 2      | 73.24%      | 73.31%       | +0.07               |
| 4      | 72.25%      | 72.40%       | +0.15               |
| 16     | 70.96%      | 68.92%       | −2.04               |

Table 5: The built-in 200-example post-training check vs. the full 1,319-example test set, same checkpoints.

| Variant                                   | 200-example eval              | Full 1,319-example eval          |
|-------------------------------------------|-------------------------------|----------------------------------|
| <code>vanilla_lora</code>                 | 73.00%                        | 72.71%                           |
| <code>lora_plus_only</code> ( $\rho=16$ ) | <b>73.50%</b> (beats vanilla) | <b>70.96%</b> (loses to vanilla) |

since the vanilla run at seed 42 happened to be its best of the three. The combined method is also markedly more consistent across seeds (spread 0.38 points vs. 1.44 for plain LoRA), though we treat this secondary observation cautiously given  $n = 3$ .

We additionally re-ran the full combined method at  $\rho \in \{2, 4\}$  (Table 4): at both sane ratios, enabling the gate gives a small, consistently-signed edge over LoRA+ alone (+0.07 to +0.15 points), while at the poorly-tuned default ( $\rho=16$ ) the gate is catastrophic (−2.04 points vs. LoRA+ alone). This suggests the gate does not itself cause the negative interaction observed in Table 1; rather, it amplifies whatever LoRA+ is already doing at a given ratio, for better or worse.

## 5.6 A methodological note: evaluation subset size can flip an effect’s sign

Every training run in our pipeline also computes a quick post-training check on the first 200 test examples. Table 5 shows this can contradict the full-set result for the identical checkpoint: `lora_plus_only` appears to beat `vanilla_lora` on 200 examples but loses on the full set. We report this because it is a general caution for GSM8K evaluation practice independent of our specific method — a subset evaluation is not simply noisier, it can invert a conclusion.

## 6 Discussion

**What is actually being claimed.** Because the gate never alters the forward computation (Section 3), its only possible effect is on which  $A, B$  values training converges to. The confirmed result is therefore narrow and specific: *a training-time-only, zero-parameter, zero-inference-cost modification that, at a corrected learning-rate ratio, yields a modest but real improvement over plain LoRA on one task and one model.* We do not claim architectural novelty, a new capability, or generality beyond what we tested.

**Why might an explicit gate help on top of an already-adaptive optimizer?** AdamW, used throughout, already performs per-parameter adaptive scaling via its gradient second-moment estimates. A natural objection is that an explicit gradient-norm gate should be redundant with this. Our ablation does not resolve this question — it shows the gate has a measurable, seed-robust effect at sane ratios, not why that effect exists beyond Adam’s own adaptivity. The  $a_{\text{weight}}$  trajectory (Section 5), which drifts over training rather than sitting at a static value, is consistent with the

gate tracking a per-layer imbalance that Adam’s per-parameter (not per-layer, not  $A$ -vs- $B$ -relative) statistics do not directly represent, but we regard this as a hypothesis for future mechanistic work, not something established here.

**Honest comparison to the state of the art.** LoRA-RITE’s verified +7.13-point GSM8K gain [LoRA-RITE, 2024] is several times our effect size, achieved through a properly derived, second-order-style optimizer correction rather than a heuristic. ALLoRA’s reported +0.3% average gain [ALLoRA, 2024] is the closer comparison in both spirit (simple, per-step) and magnitude. We position OctoLoRA’s gate as a much simpler heuristic with a smaller but real effect, not as a replacement for more principled approaches — and note that a natural revision, if this line of work continues, is to adopt LoRA-RITE’s transformation-invariance framing or ALLoRA’s normalization rather than iterate further on the current ad hoc EMA-gradient-ratio construction, both of which have demonstrated larger, independently verified gains.

## 7 Limitations

Only the headline comparison (`vanilla_lora` vs. `octo_lora_plus` at  $\rho=2$ ) has three-seed confirmation; the remaining ablation cells and ratio-sweep points are single-seed and should be treated as suggestive pending the same treatment. All results are on one task (GSM8K) and one model (Llama-3.1-8B-Instruct); we make no claim about generalization to other reasoning tasks, model families, or scales. We compare against a from-scratch reimplement of LoRA rather than a standard library (e.g. `peft`) baseline, so implementation details unrelated to the gate or LoRA+ (initialization, numerical precision) could in principle contribute to observed differences. Finally, we offer no theoretical account of why the gate’s specific construction should help; Section 6’s hypothesis is not tested here.

## 8 Conclusion

We presented OctoLoRA, a gradient-adaptive, per-layer alternative to LoRA+’s fixed learning-rate ratio for balancing LoRA’s  $A$  and  $B$  matrices. An initial ablation appeared to refute the method entirely; a subsequent hyperparameter diagnosis and sweep showed this was an artifact of an over-aggressive ratio setting, and at a corrected ratio the method yields a real, seed-confirmed 1.3-point improvement over plain LoRA on GSM8K, at zero additional inference cost or parameters. We report this result alongside its full negative-result history and situate it honestly against larger, more principled effects reported by concurrent work on the same underlying problem, as we believe this progression is at least as useful to other practitioners as the final number.

## Acknowledgments

Implementation, debugging, experiment execution, and analysis in this project were carried out with substantial assistance from Claude Code (Anthropic).

## References

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.
- Soufiane Hayou, Nikhil Ghosh, and Bin Yu. LoRA+: Efficient Low Rank Adaptation of Large Models. *arXiv preprint arXiv:2402.12354*, 2024.
- ALLoRA: Adaptive Learning Rate Mitigates LoRA Fatal Flaws. *arXiv preprint arXiv:2410.09692*, 2024.
- LoRA Done RITE: Robust Invariant Transformation Equilibration for LoRA Optimization. *arXiv preprint arXiv:2410.20625*, 2024.
- Balanced Low-Rank Adaptation: Removing Parameter Invariance to Accelerate Convergence. *arXiv preprint arXiv:2605.31484*, 2026.
- Heming Zou, Yunliang Zang, Wutong Xu, Yao Zhu, and Xiangyang Ji. FlyLoRA: Boosting Task Decoupling and Parameter Efficiency via Implicit Rank-Wise Mixture-of-Experts. *arXiv preprint arXiv:2510.08396*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- AI@Meta. Llama 3 Model Card. 2024. <https://github.com/meta-llama/llama3>